DEVELOPMENT OF AN

Ethernet-Based Vehicle Monitoring System

Using Linux Socket Programming in C

System Architecture and Implementation Design

Project platform: Pop!_OS / Linux

Communication: TCP and UDP sockets

1. Introduction

This project implements a software-based vehicle monitoring system using Linux socket programming in C. A Vehicle ECU application generates vehicle signals and sends them to a Dashboard ECU application, which receives, validates, and displays the values.

2. Objectives

- Generate vehicle speed, engine RPM, and steering angle periodically.
- Implement UDP datagram communication and TCP stream communication.
- Inspect application traffic using Wireshark.
- Evaluate normal operation and selected failure scenarios.

3. System Architecture

Component	Responsibility
Vehicle ECU	Generates vehicle data and transmits it to the dashboard.
Dashboard ECU	Receives data, validates message fields, and displays readings.
Linux sockets	Provide the application interface to UDP and TCP networking.
Wireshark	Captures and decodes packets for analysis.

4. Communication Design

UDP mode: The dashboard binds to 127.0.0.1:5005. The vehicle application sends datagrams once per second. Each datagram contains a comma-separated speed, RPM, and steering angle.

TCP mode: The dashboard listens on 127.0.0.1:5006. The vehicle establishes a connection and sends newline-terminated records over a TCP byte stream. The dashboard accumulates received bytes and processes complete lines.

5. Vehicle Data Format

Signal	Unit	Example
Vehicle speed	km/h	42
Engine RPM	rpm	1800
Steering angle	degrees	-20
Message	CSV-style text	42,1800,-20

The current application uses simulated values for demonstration; it is not connected to real vehicle sensors or an actual automotive ECU.

6. Socket Operation

Vehicle ECU	Dashboard ECU
Create socket	Create socket
Configure destination	Bind local address and port
TCP: connect / UDP: sendto	TCP: listen and accept / UDP: recvfrom
Transmit periodic messages	Receive and validate messages
Close socket	Close client and listening socket

7. Error Handling and Limitations

The programs check socket creation, address parsing, binding, and key send/receive operations. The dashboard performs basic range and format validation. UDP does not guarantee delivery. TCP provides a reliable, ordered byte stream, but the application must frame its messages.

The demonstration uses loopback networking on one Linux host. A deployment across separate ECUs would require appropriate interface addresses, routing, firewall rules, and network security.

8. Build and Execution

Build using `make`. Run `./build/dashboard_ecu udp` in one terminal and `./build/vehicle_ecu udp` in another. For TCP, run `./build/dashboard_ecu tcp` and `./build/vehicle_ecu tcp`.

9. Conclusion

The design demonstrates application-layer vehicle data exchange using Linux UDP and TCP sockets, with packet inspection performed separately using Wireshark.